

Cuarto Avance – Desarrollo de una Aplicación Web con *React + Vite y FastAPI*

No. de Ficha:	3406204
Trimestre:	03
Ambiente:	502
Competencia:	<i>React</i>
Instructor:	Jhan Hader Muñoz

Actividad Práctica: Cuarto Avance – Integración Full Stack con React + Vite, FastAPI y Base de Datos SQL

Contexto de la actividad

Una vez finalizados los requerimientos establecidos en los avances anteriores, el aprendiz deberá continuar con el desarrollo de la aplicación web construida con *React + Vite*, incorporando en esta oportunidad un *backend* desarrollado con *Python y FastAPI*, conectado a una base de datos relacional SQL.

En el tercer avance se planteó la integración del Frontend con un *Backend* desarrollado en *JavaScript* y una base de datos SQL, permitiendo implementar persistencia de información, autenticación, autorización, operaciones CRUD (*Create, Read, Update, Delete*), gestión de usuarios y control de roles.

Para este cuarto avance se deberá mantener la funcionalidad y estructura desarrollada previamente, pero reemplazando la tecnología del *Backend* por *FastAPI, framework de Python* orientado a la construcción de *APIs REST*.

La solución deberá conservar la arquitectura general del proyecto:

React + Vite → *FastAPI* → Base de Datos SQL

El aprendiz deberá demostrar que la aplicación puede comunicarse correctamente entre el Frontend y el Backend, realizar operaciones sobre la base de datos, autenticar usuarios mediante JWT (*Jason Web Token*), proteger *endpoints*, controlar permisos según el rol y gestionar la información mediante operaciones *CRUD*.

Objetivo de aprendizaje

Desarrollar la integración *Full Stack* de una aplicación web utilizando *React + Vite* como *Frontend* y *FastAPI* como *Backend*, conectados a una base de datos relacional SQL, implementando autenticación segura mediante *JWT, hashing* de contraseñas, validaciones, control de roles, protección de *endpoints*, operaciones *CRUD* y paneles diferenciados para administradores, empleados y clientes.

Requerimientos del cuarto entregable

El aprendiz deberá continuar trabajando sobre el proyecto desarrollado durante los avances anteriores.

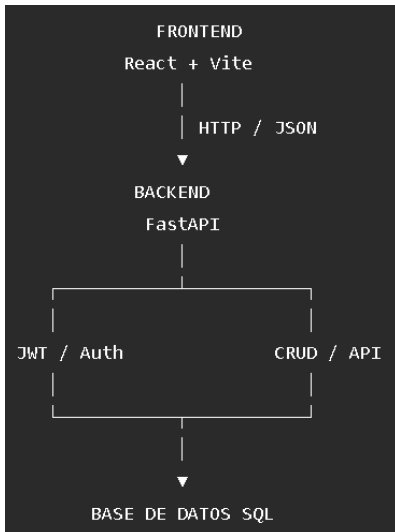
Se deberá conservar:

- La interfaz desarrollada con React + Vite.
- Los componentes reutilizables.
- Tailwind CSS.
- El formulario de registro.
- El formulario de inicio de sesión.
- El módulo de recuperación de contraseña.
- Las validaciones desarrolladas previamente.
- El Navbar.
- El Footer.
- El carrusel y demás componentes propios del proyecto.
- El botón flotante de WhatsApp.
- Las páginas y funcionalidades desarrolladas anteriormente.

El objetivo no es crear un proyecto completamente diferente, sino evolucionar el proyecto existente hacia una arquitectura Full Stack utilizando FastAPI.

1. Arquitectura tecnológica

Para este avance se deberá utilizar la siguiente arquitectura sugerida:



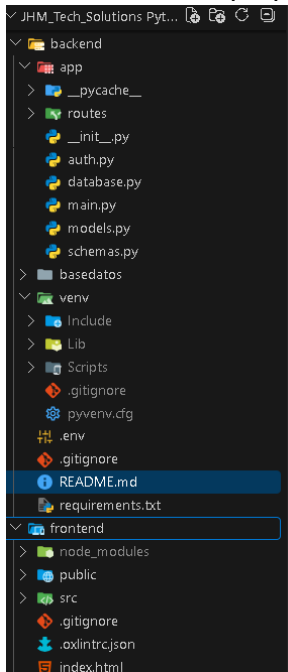
La comunicación deberá realizarse mediante solicitudes *HTTP* utilizando formatos de intercambio de información como JSON.

El *Frontend* deberá consumir los *endpoints* desarrollados en *FastAPI* mediante herramientas como *fetch*, *Axios* u otra alternativa apropiada.

2. Creación de la carpeta *Backend* con *FastAPI*

El proyecto deberá organizarse separando claramente el *Frontend* y el *Backend*.

Estructura inicial propuesta:



La estructura podrá variar de acuerdo con las características del proyecto.

3. Configuración del entorno *FastAPI*

El aprendiz deberá configurar un entorno virtual de Python para el Backend. Se recomienda crear el entorno mediante:

- `python -m venv venv`

Activación en Windows:

- `venv\Scripts\activate`

Posteriormente deberá instalar las dependencias necesarias para el funcionamiento de la API. Entre las principales tecnologías que podrán utilizarse se encuentran:

- FastAPI
- Uvicorn
- SQLAlchemy
- Pydantic
- PyMySQL o mysql-connector-python
- python-dotenv
- python-jose
- passlib/bcrypt

Las dependencias deberán quedar registradas en **requirements.txt**.

Ejemplo:

- fastapi
- uvicorn
- sqlalchemy
- pydantic
- python-dotenv
- python-jose
- passlib
- bcrypt
- pymysql

Las dependencias podrán variar de acuerdo con la implementación realizada por el aprendiz.

4. Creación de la base de datos relacional SQL

Se deberá mantener y completar la base de datos desarrollada en el avance anterior.

Como mínimo deberán contemplarse entidades relacionadas con:

- Usuarios.
- Roles.
- Permisos.
- Productos.
- Servicios.

Podrán agregarse tablas adicionales de acuerdo con las necesidades específicas del proyecto. Ejemplo conceptual:

ROLES



PRODUCTOS

SERVICIOS

La base de datos deberá permitir almacenar de manera persistente la información generada desde la aplicación.

5. Tabla de usuarios

La tabla usuarios deberá conservar los campos correspondientes al formulario desarrollado durante los avances anteriores:

- Nombre.
- Apellido.
- Tipo de documento.
- Número de documento.
- Dirección.
- Teléfono.
- Correo electrónico.
- Contraseña.
- Rol.
- Estado.

La contraseña deberá almacenarse exclusivamente mediante un mecanismo de hashing seguro. No se permitirá almacenar contraseñas en texto plano. Ejemplo conceptual:

```
usuarios
-----
id
nombre
apellido
tipo_documento
numero_documento
direccion
telefono
correo
password_hash
rol_id
estado
fecha_creacion
```

6. Modelos y esquemas en FastAPI

El *Backend* deberá diferenciar entre los modelos utilizados para interactuar con la base de datos y los esquemas utilizados para validar los datos recibidos por la API.

Se recomienda implementar:

```
models/
├─ usuario.py
├─ rol.py
├─ producto.py
└─ servicio.py

schemas/
├─ usuario.py
├─ auth.py
├─ producto.py
└─ servicio.py
```

Los esquemas deberán permitir validar:

- Tipos de datos.
- Campos obligatorios.
- Longitudes.
- Formatos.
- Correos electrónicos.
- Contraseñas.
- Información de productos.
- Información de servicios.

7. Conexión entre FastAPI y la base de datos

El *Backend* deberá establecer una conexión funcional con la base de datos SQL.

La configuración de conexión deberá realizarse mediante variables de entorno y no deberá exponerse directamente dentro del código fuente.

Ejemplo conceptual:

```
DB_HOST
DB_PORT
DB_NAME
DB_USER
DB_PASSWORD
SECRET_KEY
```

El archivo `.env` deberá mantenerse fuera del repositorio público cuando contenga información sensible.

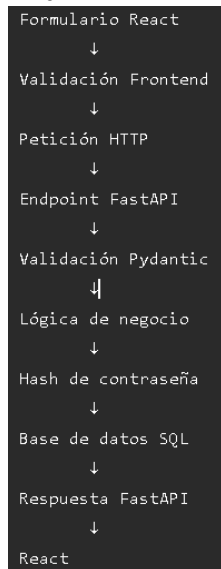
La arquitectura deberá quedar:

```
React + Vite
|
▼
FastAPI
|
▼
SQLAlchemy / Driver SQL
|
▼
Base de datos SQL
```

8. Conexión entre *Frontend*, *Backend* y Base de Datos

El formulario desarrollado en React deberá dejar de funcionar únicamente como interfaz visual y deberá comunicarse con FastAPI.

Flujo esperado:



El *Backend* deberá validar nuevamente la información recibida, independientemente de las validaciones realizadas en el *Frontend*.

9. Registro de clientes

El formulario de registro desarrollado anteriormente deberá conectarse con FastAPI.

Cuando un cliente complete correctamente el formulario:

1. *React* deberá validar los datos.
2. *React* enviará una petición *HTTP*.
3. *FastAPI* recibirá la información.
4. *FastAPI* validará los datos.
5. Se verificará que el correo o documento no estén duplicados.
6. La contraseña será convertida en un hash seguro.
7. La información será almacenada en la base de datos.
8. *FastAPI* devolverá una respuesta *JSON*.
9. *React* mostrará el resultado al usuario.

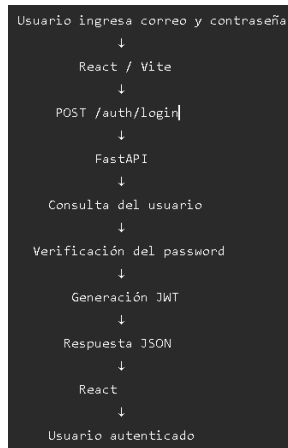
Ejemplo conceptual de petición:

POST /api/usuarios/registro

La contraseña original no deberá almacenarse en la base de datos.

10. Inicio de sesión

El módulo de inicio de sesión desarrollado con React deberá conectarse con FastAPI. Flujo será:



FastAPI deberá verificar las credenciales y generar un *JSON Web Token (JWT)* cuando sean correctas.

11. Autenticación mediante *JWT*

El sistema deberá utilizar *JWT* para mantener la autenticación de los usuarios. El token deberá contener información necesaria para identificar al usuario y controlar su acceso.

De manera conceptual:

```
{
  "sub": "usuario@correo.com",
  "role": "cliente",
  "exp": "fecha_expiracion"
}
```

El *token* deberá ser enviado desde *React* en las solicitudes que requieran autenticación. Ejemplo:

Authorization: Bearer TOKEN

FastAPI deberá verificar:

- Existencia del token.
- Validez del token.
- Firma.
- Expiración.
- Usuario asociado.
- Rol cuando corresponda.

12. Control de roles

La aplicación deberá conservar como mínimo los siguientes roles:

- Administrador
- Empleado
- Cliente

Los roles deberán determinar las funcionalidades disponibles para cada usuario.

Administrador:

- Usuarios.
- Productos.
- Servicios.
- Estados.
- Información administrativa.

Empleado:

- Podrá acceder únicamente a las funcionalidades relacionadas con las actividades asignadas.

Cliente:

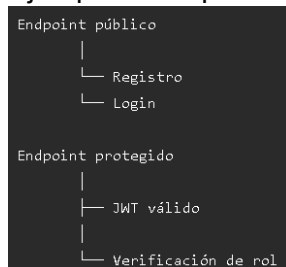
- Podrá acceder a las funcionalidades correspondientes a un usuario cliente.

El control deberá realizarse tanto en el *Frontend* como en el *Backend*, pero la autorización definitiva deberá ser responsabilidad del *Backend*.

13. Protección de *endpoints*

Los *endpoints* que requieran autenticación deberán estar protegidos mediante JWT. *FastAPI* deberá utilizar dependencias para verificar la identidad del usuario.

Ejemplo conceptual:



Un usuario no deberá poder acceder manualmente a endpoints restringidos modificando únicamente la URL desde el navegador.

14. Endpoints de la API

El aprendiz deberá crear los endpoints necesarios para gestionar las funcionalidades del proyecto.

Usuarios:

- POST /api/usuarios/registro
- POST /api/auth/login
- GET /api/usuarios
- GET /api/usuarios/{id}
- PUT /api/usuarios/{id}
- PATCH /api/usuarios/{id}/estado
- DELETE /api/usuarios/{id}

Productos:

- GET /api/productos
- POST /api/productos
- GET /api/productos/{id}
- PUT /api/productos/{id}
- DELETE /api/productos/{id}

Servicios, cuando corresponda:

- GET /api/servicios
- POST /api/servicios
- GET /api/servicios/{id}
- PUT /api/servicios/{id}
- DELETE /api/servicios/{id}

La estructura de las rutas podrá variar de acuerdo con la organización del proyecto.

15. Métodos HTTP

Los *endpoints* deberán demostrar el uso de los principales métodos HTTP:

```
GET    → Consultar
POST   → Crear
PUT    → Actualizar
PATCH → Modificar parcialmente
DELETE → Eliminar
```

El aprendiz deberá probar cada operación utilizando Postman u otra herramienta equivalente.

16. Implementación de operaciones CRUD

El sistema deberá implementar operaciones CRUD:

```
CREATE → Crear registros
READ   → Consultar registros
UPDATE → Actualizar registros
DELETE → Eliminar registros
```

Para los usuarios deberá existir la posibilidad de:

- Consultar.
- Crear.
- Editar.
- Actualizar.
- Cambiar estado.
- Eliminar.

Se recomienda implementar:

- Activo
- Inactivo

para conservar información histórica cuando la lógica del proyecto así lo requiera.

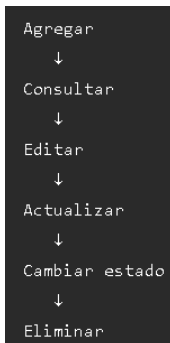
17. Panel de administración

Se deberá desarrollar un Panel de Administrador utilizando:

- *React*.
- *Vite*.
- *Tailwind CSS*.
- *FastAPI*.

El panel deberá permitir gestionar la información de los usuarios y demás entidades o módulos definidas en el proyecto.

Como mínimo:



El acceso deberá estar protegido mediante autenticación y autorización. Únicamente los usuarios con rol de administrador podrán acceder al panel administrativo.

18. Panel de empleado

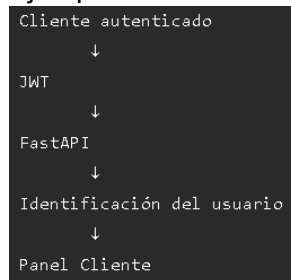
- Se deberá desarrollar un Panel de Empleado.
- El panel deberá mostrar únicamente las funcionalidades correspondientes al rol del empleado.

- El empleado no deberá disponer automáticamente de los mismos privilegios del administrador.
- El Backend deberá validar el rol antes de permitir operaciones restringidas.

19. Panel de cliente

- Se deberá desarrollar un Panel de Cliente.
- El cliente deberá poder acceder a las funcionalidades establecidas para este rol.
- El sistema deberá identificar correctamente al usuario autenticado mediante la información asociada al JWT.

Ejemplo:



20. Usuario autenticado en el *Navbar*

Después de iniciar sesión correctamente, *React* deberá mostrar el nombre del usuario autenticado en el *Navbar*.

Ejemplo:

Inicio | Productos | Servicios | Bienvenido, Juan | Cerrar sesión

- La información deberá corresponder al usuario que inició sesión.
- Cuando el usuario cierre sesión, el *Navbar* deberá actualizarse.

21. Validaciones en tiempo real

Las validaciones desarrolladas durante el segundo avance deberán mantenerse y fortalecerse.

Los formularios deberán controlar:

- Campos obligatorios.
- Longitud mínima.
- Longitud máxima.
- Tipo de dato.
- Expresiones regulares.
- Correo electrónico.
- Número de documento.
- Teléfono.

- Contraseña.
- Confirmación de contraseña.
- Caracteres permitidos.

Las validaciones deberán ejecutarse en:

React +FastAPI

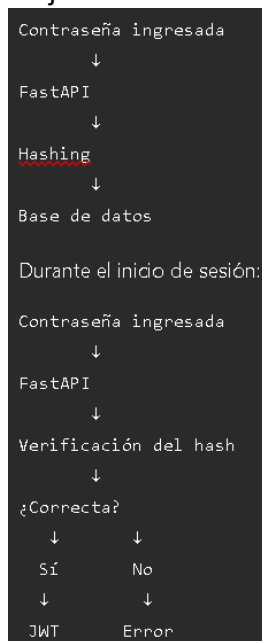
La validación del Backend será obligatoria aunque el Frontend haya validado previamente los datos.

22. Seguridad de contraseñas

Está prohibido almacenar las contraseñas directamente en la base de datos. FastAPI deberá utilizar un mecanismo seguro de hashing para:

- Generar el hash.
- Almacenar el hash.
- Verificar la contraseña durante el login

Flujo deberá ser:



23. Variables de entorno

La información sensible deberá gestionarse mediante variables de entorno. Se podrán utilizar variables como:

```
DATABASE_URL=
SECRET_KEY=
ALGORITHM=
ACCESS_TOKEN_EXPIRE_MINUTES=
```

No se deberán publicar:

- Contraseñas de la base de datos.
- Claves secretas.
- Tokens.
- Credenciales.
- Información privada.

24. Componente flotante de WhatsApp

Se deberá conservar el componente reutilizable de WhatsApp desarrollado en los avances anteriores. El componente deberá:

- Ser reutilizable.
- Mantenerse visible.
- Tener una posición fija.
- Utilizar el enlace de WhatsApp correspondiente.
- Mantener coherencia con el diseño.
- Permitir contactar fácilmente con la empresa.

Ejemplo de componente:

WhatsAppButton.jsx

Este componente deberá continuar funcionando independientemente del Backend.

25. Documentación automática de *FastAPI*

Una de las características que deberá aprovechar el aprendiz es la documentación automática proporcionada por FastAPI. Al ejecutar el servidor, deberá poder acceder a:

<http://127.0.0.1:8000/docs>

Esta interfaz permitirá visualizar y probar los *endpoints* mediante Swagger UI. El aprendiz deberá utilizar esta documentación como evidencia del funcionamiento de la API.

26. Pruebas con *Postman*

Se deberán realizar pruebas de los endpoints utilizando *Postman* u otra herramienta equivalente.

Como mínimo deberán evidenciarse:

- Registro de usuario.
- Inicio de sesión.
- Generación de JWT.
- Consulta de usuarios.
- Consulta individual.

- Actualización de usuarios.
- Cambio de estado.
- Eliminación.
- Consulta de productos.
- Creación de productos.
- Actualización de productos.
- Eliminación de productos.
- Endpoints protegidos.
- Control de roles.
- Respuestas correctas.
- Respuestas de error.

Deberán evidenciarse los métodos:

- **GET**
- **POST**
- **PUT**
- **PATCH**
- **DELETE**

Entregables

El aprendiz deberá presentar:

1. Proyecto completo de React + Vite desarrollado durante los avances anteriores.
2. Backend desarrollado con FastAPI.
3. Entorno virtual de Python correctamente configurado.
4. Archivo requirements.txt.
5. Base de datos SQL.
6. Script SQL de creación de la base de datos y sus tablas.
7. Conexión entre React, FastAPI y la base de datos.
8. Formulario de registro conectado con la base de datos.
9. Inicio de sesión conectado con FastAPI.
10. Autenticación mediante JWT.
11. Contraseñas almacenadas mediante hashing seguro.
12. Endpoints de la API.
13. Validaciones Frontend y Backend.
14. Protección de endpoints.
15. Control de roles.
16. Panel de administrador.
17. Panel de empleado.
18. Panel de cliente.
19. Operaciones CRUD.
20. Usuario autenticado visible en el Navbar.

21. Componente flotante de WhatsApp.
22. Documentación automática de FastAPI mediante Swagger.
23. Pruebas realizadas mediante Postman.
24. Evidencias de las solicitudes GET, POST, PUT/PATCH y DELETE.
25. Evidencias de autenticación mediante JWT.
26. Evidencias de control de roles.
27. Evidencias de la conexión con la base de datos.
28. Código fuente completo y organizado.
29. Archivo `.env` configurado localmente.
30. Evidencias mediante capturas de pantalla.

Resultado esperado

Al finalizar el cuarto avance, el aprendiz deberá contar con una aplicación web Full Stack funcional, en la cual el *Frontend* desarrollado con *React + Vite* se comuniquen con un *Backend* desarrollado mediante *FastAPI*, y este a su vez se conecte con una base de datos relacional *SQL*.

La aplicación deberá permitir que los usuarios se registren, inicien sesión y accedan a funcionalidades de acuerdo con su rol.

La autenticación deberá implementarse mediante *JWT*, las contraseñas deberán almacenarse mediante *hashing* seguro, y los endpoints que contengan información o funcionalidades restringidas deberán estar protegidos.

La solución deberá incorporar operaciones *CRUD*, validaciones, control de roles, paneles diferenciados, integración con la base de datos, pruebas mediante Postman y documentación automática de la API mediante Swagger.

El resultado final deberá demostrar la evolución del proyecto desarrollado durante los avances anteriores, manteniendo el diseño y funcionalidades existentes, pero reemplazando el Backend basado en Node.js por una arquitectura basada en *Python + FastAPI*.